

UCC 152-2

Introduction to security in DevOps

Workshop 4: Securing containers,
dependencies, and deployment artifacts

Cloud Computing Professional License

University of the Mountains

By **Engineer Willy Damtchou**

July 2026

Part 1: General framework of workshop 4

1.1 General context of the workshop

As part of the UCC152-2 Introduction to Security in DevOps course, Workshop 4 focuses on container and dependency security within the DigiBank project. Having progressively built, observed, and hardened the application according to the principles already covered, it is now necessary to examine the application not just as code or as a running system, but as a complete software artifact—that is, as a collection of images, third-party libraries, configuration files, build mechanisms, and packaging elements, all of which can potentially introduce vulnerabilities. This transition directly corresponds to Session 4 of the course, which covers risks associated with container images, container security principles, software dependency analysis, identification of known vulnerabilities in libraries, and best practices for updating, reducing the attack surface, and hardening.

This step is crucial because an application can be properly structured, functional, and even tested, yet still be vulnerable due to what it includes during build and execution. An overly large base image, a vulnerable library, a secret left in a configuration, a container running with excessive privileges, or a lack of automated pipeline controls are all enough to severely degrade the system's security posture. Workshop 4 is specifically designed to highlight this often underestimated dimension of modern software risk.

1.2 Pedagogical positioning of the workshop

From a pedagogical standpoint, Workshop 4 shifts the student's perspective on security from a focus on code and application behavior to a broader perspective encompassing delivery and execution artifacts. Previous workshops may have emphasized the quality of the application base, code analysis, or observation of application behavior in operation. Here, the focus shifts to what DigiBank actually imports, builds, packages, exposes, and executes within its technical environment.

This shift in mindset is fundamental to a DevSecOps approach. The course demonstrates that risk doesn't stem solely from the business logic written by the team, but also from third-party dependencies, container images, configurations, secrets, build tools, and the delivery pipeline itself. Therefore, the pedagogical objective of Workshop 4 isn't simply to add another scanner to the project, but to teach students to consider containers



and dependencies as objects of analysis in their own right, and then to integrate their control into a continuous security mechanism.

1.3 Purpose of workshop 4

The goal of Workshop 4 is to conduct a security analysis of DigiBank's containers and dependencies, and then to correct the identified weaknesses through remediation, hardening, and automation actions within the CI/CD pipeline. In other words, the expected outcome of this workshop is not a completely new application, but a version of DigiBank with better control over its technical artifacts, a more streamlined package, more secure dependencies, and more rigorous security checks prior to integration or deployment.

It is therefore important to understand that workshop 4 is neither a simple document review nor a superficial demonstration of tools. It is a methodical exercise aimed at identifying at-risk components, understanding why they pose problems in the DigiBank context, proposing corrections consistent with the existing architecture, and then integrating these controls into a CI/CD approach compatible with the principles of the course.

1.4 DevSecOps Workshop Logic

This workshop provides a very concrete illustration of a central idea of DevSecOps: security doesn't stop at the source code or application testing, but must permeate the entire software development chain. In a modern environment, dependencies, base images, container layers, configuration secrets, and build scripts all directly contribute to the system's attack surface. Ignoring them would be tantamount to assuming that software is secure as long as its business logic is correct, a notion explicitly refuted in session 4.

Workshop 4 thus embodies a form of continuity with the Shift Left approach. It's no longer just about shifting the analysis to the code, but also to the packaging and delivery artifacts. Scanning dependencies before release, inspecting Dockerfile build choices, reducing unnecessary components in the final image, avoiding embedded secrets, and crashing the pipeline in case of critical vulnerabilities all stem from this same logic: addressing the risk as early as possible, as close as possible to its point of introduction in the technical chain.



1.5 Why DigiBank is a good platform for a workshop on containers and dependencies

The DigiBank project is a particularly relevant case study for a workshop on container and dependency security, as it embodies several characteristics typical of modern applications. The case study presents DigiBank as a digital banking application built around a web interface, a backend API, a database, application services, and a deployment strategy based on containers and a CI/CD pipeline. This framework provides concrete material for analysis, as it combines business logic, third-party components, configuration, build artifacts, and delivery logic.

DigiBank's value also lies in the fact that the case deliberately incorporates realistic vulnerabilities: legacy dependencies, overly broad base images, secrets still present in certain configurations, incomplete hardening, and insufficient security integration within the automated chain. These elements allow for working on a credible case without resorting to an artificially simplified exercise. In Workshop 4, DigiBank is therefore not simply an application to be run; it serves as a tool for analyzing the risks associated with what a modern application actually incorporates and executes.

1.6 Definition of container and dependency analysis within the workshop

In this workshop, container and dependency analysis refers to the structured examination of external components, libraries, images, build configurations, and deployment artifacts used by DigiBank, with the goal of identifying known vulnerabilities, poor packaging practices, or technical choices that unnecessarily expand the attack surface. This analysis is therefore not limited to verifying that the application builds; it seeks to determine whether what DigiBank imports and executes introduces additional security risks.

This detail is important because an application can be compileable, testable, and even functionally consistent while still carrying vulnerable components or an insufficiently hardened image. The analysis expected in Workshop 4 must therefore cover both declared and transitive software dependencies, files such as the Dockerfile or docker-compose.yml, runtime parameters, the possible presence of secrets in the configuration, and the pipeline's ability to automatically check these elements.

1.7 General objective of the workshop

The overall objective of Workshop 4 is to detect, analyze, prioritize, remediate, and automate security risks related to containers and dependencies in the DigiBank project, building upon the concepts covered in Session 4 of the course and a DevSecOps approach focused on continuous security. This objective requires students to be able not only to run scanning or inspection tools, but also to interpret their results, distinguish truly critical issues, modify the affected artifacts without breaking the project, and then demonstrate that essential controls can be replayed in a CI/CD pipeline.

The workshop is therefore not simply about producing reports. It aims to transform these reports into understanding, and then this understanding into concrete remediation and continuous monitoring. It is in this sense that it fully aligns with the DevSecOps approach presented in the course.

1.8 Specific Objectives

At the end of this workshop, the student should be able to:

- **A4.1:** Explain the security risks associated with container images and third-party libraries in DigiBank.
- **A4.2:** Identify the DigiBank files, artifacts and configurations that participate in the containerized construction and execution of the application.
- **A4.3:** Analyze the project's software dependencies to identify old, sensitive, or vulnerable components.
- **A4.4:** Examine the Dockerfile, the produced image, and the execution parameters to identify choices that increase the attack surface.
- **A4.5:** Correct the weaknesses identified through update actions, reduction of attack surface and hardening.
- **A4.6:** Technically justify the remediations carried out on DigiBank from a security point of view.
- **A4.7:** Implement a basic CI/CD pipeline to automatically replay the main checks on dependencies and containerized artifacts.
- **A4.8:** Produce technical deliverables outlining the analysis, corrections made, and evidence of final validation.



1.9 Scope of the analysis in workshop 4

The scope of this workshop covers all weaknesses or vulnerabilities that can reasonably be observed in the dependencies, images, packaging files, runtime configurations, and automation mechanisms associated with DigiBank. This includes, but is not limited to, Maven dependencies, transitive dependencies, pom.xml files, the Dockerfile, any docker-compose.yml files, application configurations that may contain secrets, metadata of the built image, and GitHub Actions workflows related to security controls.

However, the primary objective of this workshop is not to repeat the detailed analysis of the source code or the in-depth observation of the dynamic behavior of endpoints during execution. These aspects fall primarily under the logic already addressed in the other workshops. This delimitation is important because it allows us to clearly distinguish the analysis of build and delivery artifacts from the internal analysis of the code or the external analysis of runtime behavior.

1.10 Types of vulnerabilities targeted

Within the DigiBank framework, the vulnerabilities or weaknesses sought during this workshop may include:

- Outdated or known vulnerable dependencies.
- Basic images that are too wide, too old, or unnecessarily exposed.
- Unnecessary components embedded in the final image, increasing the attack surface.
- Secrets or sensitive settings present in configuration files or versioned artifacts.
- Containers run with excessive privileges or without minimal hardening precautions.
- Ports, files, or execution mechanisms exposed without clear justification.
- A lack of automated control in the pipeline over dependencies and containerization artifacts.

It should be noted that some of these weaknesses do not necessarily imply immediate exploitation, as has already been demonstrated. The workshop's role is precisely to learn to recognize signs of risk before they become more costly incidents to correct, which directly aligns with the continuous safety logic advocated in the course.

1.11 Types of vulnerabilities targeted

The tools selected for workshop 4 cover development, dependency analysis, containerization, and pipeline automation. They should be presented as complementary instruments, each addressing a specific part of DigiBank's technical chain.

- **IntelliJ IDEA** is used to open, read, modify, and rebuild the DigiBank project.
- **JDK 17** allows the application to be compiled and run.
- **Maven** handles the build, dependencies, and analysis plugins.
- **Spring Boot** provides the application framework used by DigiBank.
- **Docker** allows you to build and run the containerized image of the application.
- **Docker Compose** makes it easy to set up a reproducible environment combining application and database.
- **OWASP Dependency-Check** helps identify known vulnerabilities in libraries used by the project.
- **Trivy** analyzes Docker images to detect vulnerabilities in system packages, embedded libraries, misconfigurations, and secrets potentially present in containers.
- **Git** allows you to track changes made to build, configuration, and hardening files.
- **GitHub** hosts the repository and centralizes the analyzed code.
- **GitHub Actions** automates building, dependency scans, image building, and publishing proof artifacts in a CI/CD logic.

Depending on the level of detail chosen in the rest of the workshop, other container scanning tools can be added as a complement, but the basic structure remains as follows: Maven for the build chain and dependencies, Docker for the containerized artifact, and GitHub Actions for the automation of checks.

1.12 Expected result at the end of part 1

At the end of this first part, the student should understand why Workshop 4 exists, what securing containers and dependencies truly means in a project like DigiBank, and how this step extends the DevSecOps logic studied in the course. They should be able to explain why images, third-party libraries, packaging files, configuration secrets, and the CI/CD pipeline are integral parts of a modern application's attack surface.

They must also be able to explain the exact scope of the work, which risk categories will be investigated, which tools will be used, and why DigiBank is a relevant case study for



this practical application. The next phase can then move into the actual operational work: preparing the environment, verifying the tools, launching the project, checking the initial artifacts, and setting up the practical framework for analyzing containers and dependencies.

Part 2: Practical preparation of the DigiBank container and dependency analysis environment

In this second part of Workshop 4, the goal is not yet to immediately delve into the detailed identification of dependency vulnerabilities or the complete hardening of DigiBank containers, but rather to prepare a sufficiently clean, consistent, and reproducible analysis environment so that the security checks performed on the artifacts are meaningful. This step is fundamental because a workshop dedicated to container and dependency security is not about mechanically running a few scan commands on a vaguely open project. On the contrary, it's necessary to start with a functional and already reasonably stable version of DigiBank, verify the build artifacts actually used, check the packaging files, clarify the dependency structure, and put in place the elements that will then allow for credible analysis, reasoned remediation, and realistic automation within the CI/CD pipeline.

This approach is fully consistent with the course progression. Workshop 1 enabled the construction of a modular, executable, and industrializable application base. Workshop 2 taught participants to examine code, configuration, and dependencies from a SAST perspective. Workshop 3 shifted the focus to the actual behavior of the application during execution. Workshop 4 now extends this path by examining the artifacts that truly transport the application into its technical environment: third-party libraries, Docker image, build configuration, injected or embedded secrets, and the future continuous monitoring pipeline.

2.1 Role of this phase in the workshop

Before being able to usefully detect vulnerabilities in dependencies or in the DigiBank image, the student must ensure that the project still rebuilds correctly, that the actual packaging files are present, that the Maven dependency graph is readable, that the Spring configuration is based on appropriate variables, and that the Dockerfile is not simply an artifact that has been forgotten or is inconsistent with the current state of the application. This requirement is important because a security analysis of artifacts quickly becomes misleading if one is working on a partially broken repository, on unused files, or on an image that does not truly correspond to the application being tested.

This phase therefore has a threefold purpose. First, it restores DigiBank to a clean technical state before analysis. Second, it prepares the main components for Workshop 4, including the parent pom.xml file, the Dockerfile, the docker-compose.yml file, the application*.yml files, the environment variables, and the directories where the reports will be stored. Finally, it establishes the conditions for workshop reproducibility, so that dependency analyses, image rebuilds, and initial security checks can be consistently replayed, both locally and later in GitHub Actions.

2.2 Software Prerequisites

The student must start from a workstation already operational for the Java, Maven, Spring Boot and Docker environment used in DigiBank, in accordance with the standardized technical choices from workshop 1, then reused in workshops 2 and 3. It is therefore not necessary here to explain at length the complete installation of the basic environment; on the other hand, it is necessary to verify that the components required for workshop 4 are indeed available, because the security of the containers and dependencies relies on the reconstruction of the project, the execution of Maven tools, the building of Docker images and the inspection of the produced artifacts.

The components to install or check are as follows:

- Java JDK 17.
- Maven 3.9.x or compatible version.
- IntelliJ IDEA Community or Ultimate.
- Git.
- Docker Desktop or Docker Engine with Docker Compose.
- A modern browser, useful for certain local checks and viewing HTML reports.
- Access to GitHub is required if you wish to prepare the CI/CD integration of controls from this phase.

The minimum verification commands to execute in a terminal are as follows:

```
Java -version  
mvn -version  
git --version
```

```
docker --version  
  
docker compound version  
  
psql --version
```

If any of these commands fail, the problem must be fixed before proceeding. A workshop on dependencies and containers quickly becomes confusing if a Maven build failure, a plugin failure, or Docker unavailability is then mistakenly interpreted as a significant security signal.

2.3 Tools selected for the workshop environment 4

In this section, it is important not to present the tools as isolated elements, but as building blocks of a single chain for analyzing software artifacts. Each tool has a specific function in the preparation strategy for workshop 4, and its role must be understood in relation to the others.

The tools selected are as follows:

- **IntelliJ IDEA** allows you to open DigiBank, check the consistency of the multi-module Maven project, edit pom.xml, Dockerfile, docker-compose.yml files and Spring configurations.
- **Maven** allows you to rebuild the application, produce the Java artifact, view direct and transitive dependencies, and run OWASP Dependency-Check.
- **Spring Boot** provides the application framework to be containerized and properly linked to environment variables.
- **Docker** allows you to build the DigiBank image, observe its general structure and prepare for future hardening controls.
- **Docker Compose** makes it easy to set up a reproducible environment combining the application and PostgreSQL.
- **OWASP Dependency-Check** is used to identify known vulnerabilities in libraries used by the project.
- **Trivy** analyzes Docker images to detect vulnerabilities in system packages, embedded libraries, misconfigurations, and secrets potentially present in containers.

- **Git** and **GitHub** are used to track configuration, packaging, and hardening changes, and then to prepare for future integration with GitHub Actions.
- **GitHub Actions** will play the role of automation support to replay the build, dependency scan and image build in a CI/CD logic.

Within the pedagogical framework of this course, it is essential to understand that Maven and Docker are the two cornerstones of the preparation phase. Maven represents the entry point for dependencies and the build cycle, while Docker represents the entry point for packaging and the delivery artifact. Workshop 4 arises precisely from the intersection of these two dimensions.

2.4 Clean reopening of the DigiBank project

The project used for workshop 4 must be the version from the previous workshops; that is, a version already built, analyzed, and partially corrected, and not an improvised or arbitrarily modified base. The course and the progression of the practical sessions clearly demonstrate that the workshops follow a logical sequence: workshop 4 does not start from a raw project, but from a DigiBank that is already structured, testable, and sufficiently stable to allow us to examine its contents and delivery method.

In IntelliJ IDEA, the recommended procedure remains as follows:

1. Open IntelliJ IDEA.
2. Choose Open.
3. Select the root folder digibank-parent.
4. Verify that the project is correctly recognized as a multi-module Maven project.
5. Wait for the project to be fully indexed.
6. Open the Maven tab.
7. Click on Reload All Maven Projects.
8. Check for the presence of the common-module, customer-module, account-module, transfer-module and digibank-web modules.
9. Check that no blocking Maven errors appear in the IDE.

At this stage, a control build must be executed from the project root:

```
CD digibank-parent  
mvn clean install
```



This command retains the same structuring role here as in workshops 2 and 3. It confirms that the version of DigiBank submitted to the analysis of dependencies and containers still compiles, that the essential tests pass, and that the student is not about to analyze artifacts produced by a project already broken upstream.

2.5 Git backup before instrumenting workshop 4

As in previous workshops, it is recommended to create a Git restore point before adding files specific to container and dependency analysis, or before modifying existing packaging artifacts. This practice remains consistent with the DevSecOps logic of traceability, reproducibility, and returning to a stable project state.

Execute :

```
git status  
  
git add .  
  
git commit -m "Baseline before DAST environment setup"
```

This commit will clearly distinguish the state of DigiBank before the instrumentation of workshop 4 and the state of the project after the addition of the configurations, scripts, reports, and containerization artifacts related to this new phase. It will be particularly useful when the student needs to justify the changes made to the Dockerfile, Maven dependencies, or execution configuration.

2.6 Verification of the Spring configuration before artifact analysis

Before delving into dependency and container analysis, it's crucial to ensure that DigiBank's Spring Boot configuration remains compatible with clean local execution, reasonable containerized execution, and future execution in a CI/CD pipeline. Workshop 2 already emphasized the separation between versioned configuration and sensitive values, and Workshop 3 reiterated this principle for DAST. Therefore, there's no need to fully

re-demonstrate it here, but it's essential to verify that this separation is maintained before building an image or publishing an artifact.

The file `digibank-web/src/main/resources/application.yml` can maintain a structure of this type:

```
spring :  
  
  application :  
  
    name : digibank  
  
  profiles :  
  
    active : ${SPRING_PROFILES_ACTIVE:dev}  
  
  jpa :  
  
    hibernate :  
  
      ddl-auto : validate  
  
    show-sql : false  
  
    open-in-view : false  
  
    properties :  
  
      hibernate :  
  
        SQL format : true  
  
      jdbc :  
  
        time zone : UTC  
  
  thymeleaf :  
  
    cache : true
```

```
SQL :

  init :

    mode : never


server :

port : ${SERVER_PORT:8080}

error :

  include-message : never

  include-binding-errors : never

  include-stacktrace : never


springdoc :

  api-docs :

    enabled : false

  swagger-ui :

    enabled : false


logging :

  level :

    root : INFO

    org.springframework.web : INFO

    org.hibernate.SQL : INFO

    org.hibernate.type.descriptor.sql.BasicBinder : INFO
```

```
error :  
  
  include-message : never  
  
  include-binding-errors : never  
  
  include-stacktrace : never
```

File digibank-web/src/main/resources/application-dev.yml

```
spring :  
  
  datasource :  
  
    url : ${SPRING_DATASOURCE_URL}  
  
    username : ${SPRING_DATASOURCE_USERNAME}  
  
    password : ${SPRING_DATASOURCE_PASSWORD}  
  
    driver-class-name : org.postgresql.Driver  
  
  jpa :  
  
    hibernate :  
  
      ddl-auto : validate  
  
    show-sql : false  
  
    open-in-view : false  
  
    properties :  
  
      hibernate :  
  
        SQL format : true  
  
      jdbc :  
  
        time zone : UTC
```

```
flyway :  
  enabled : true  
  baseline-on-migrate : true  
  locations : classpath:db/migration/postgresql  
  default-schema : public  
  schemas : public
```

File digibank-web/src/main/resources/application-test.yml

```
spring :  
  datasource :  
    url : jdbc:h2:mem:digibanktestdb  
    driver-class-name : org.h2.Driver  
    username : sa  
    password :  
  
  jpa :  
    hibernate :  
      ddl-auto : create-drop  
      show-sql : false  
  
  flyway :  
    enabled : true  
    baseline-on-migrate : true
```

```
locations : classpath:db/migration/h2
```

This configuration has several advantages in the context of workshop 4. It avoids hardcoding secrets into versioned files, it makes the application compatible with parameter injection via Docker or GitHub Actions, and it reduces the risk of the final image containing identifiers directly copied into its layers. This is a crucial point, as the course explicitly emphasizes that a container should not become a vector for the unintentional dissemination of secrets.

2.7 Verification and preparation of the parent pom.xml file for workshop 4

In workshop 4, the parent pom.xml plays a particularly important role, as it serves as the primary entry point for dependency analysis. Workshop 2 already demonstrated how to centralize versions and integrate OWASP Dependency-Check. The goal here is not to repeat that entire demonstration, but to verify that DigiBank's Maven parent is indeed ready to support a clear dependency graph and reproducible control of third-party components.

The properties section should at least centralize the major versions of the project and the analysis plugin, for example:

```
<properties>

<project.build.sourceEncoding> UTF-8
</project.build.sourceEncoding>

<project.reporting.outputEncoding> UTF-8
</project.reporting.outputEncoding>

<java.version> 21 </java.version>

<spring.boot.version> 3.4.3 </spring.boot.version>

<springdoc.version> 2.8.5 </springdoc.version>

<cucumber.version> 7.18.0 </cucumber.version>

<junit.version> 5.10.2 </junit.version>
```

```
<maven.compiler.plugin.version> 3.13.0
</maven.compiler.plugin.version>

<envfile.path> ${session.executionRootDirectory}/.env
</envfile.path>

<sonar.maven.plugin.version> 3.11.0.3922
</sonar.maven.plugin.version>

<sonar.projectKey> ${env.SONAR_PROJECT_KEY} </sonar.projectKey>

<sonar.projectName> ${env.SONAR_PROJECT_NAME}
</sonar.projectName>

<sonar.dependencyCheck.htmlReportPath>
target/dependency-check-report.html
</sonar.dependencyCheck.htmlReportPath>

<sonar.dependencyCheck.jsonReportPath>
target/dependency-check-report.json
</sonar.dependencyCheck.jsonReportPath>

<sonar.dependencyCheck.reportPath>
target/dependency-check-report.xml
</sonar.dependencyCheck.reportPath>

<sonar.pitest.reportPaths> target/pit-reports/mutations.xml
</sonar.pitest.reportPaths>

<sonar.token> ${env.SONAR_TOKEN} </sonar.token>

<sonar.login> ${env.SONAR_TOKEN} </sonar.login>

<sonar.host.url> ${env.SONAR_HOST_URL} </sonar.host.url>

<sonar.organization> ${env.SONAR_ORGANIZATION}
</sonar.organization>

<dependency-check.version> 10.0.4 </dependency-check.version>
```

```

<pitest.version> 1.17.0 </pitest.version>

<pitest.junit5.plugin.version> 1.2.1
</pitest.junit5.plugin.version>

<surefire.version> 3.2.5 </surefire.version>

<nvd.api.key> ${env.NVD_API_KEY} </nvd.api.key>

<!-- JaCoCo / SpotBugs / PMD / Checkstyle versions -->

<jacoco.version> 0.8.12 </jacoco.version>

<spotbugs.maven.plugin.version> 4.8.6.4
</spotbugs.maven.plugin.version>

<pmd.maven.plugin.version> 3.21.0 </pmd.maven.plugin.version>

<checkstyle.maven.plugin.version> 3.5.0
</checkstyle.maven.plugin.version>

<jackson-bom.version> 2.18.9 </jackson-bom.version> <!-- or
newer -->

<postgresql.version> 42.7.13 </postgresql.version>

<commons-lang3.version> 3.18.0 </commons-lang3.version>

<swagger-ui.version> 5.32.7 </swagger-ui.version>

</properties>

```

The dependency-check-maven plugin must be present in the build, with a configuration similar to this one:

```

<plugin>

  < groupId > org.owasp </ groupId >

  < artifactId > dependency-check-maven </ artifactId >

```

```
< version > ${dependency-check.version} </ version >

< configuration >

    <format> HTML </format>

    < failBuildOnCVSS > 7 </ failBuildOnCVSS >

    < skipProvidedScope > true </ skipProvidedScope >

    < skipTestScope > true </ skipTestScope >

    < nvdApiKey > ${env.NVD_API_KEY} </ nvdApiKey >

    < ossindexAnalyzerEnabled > false </
ossindexAnalyzerEnabled >

    <deletionFiles>

        < suppressionFile > dependency-check-suppressions.xml
</ suppressionFile >

    </suppressionFiles>

</configuration>

< executions >

    < execution >

        <phase> verify </phase>

        <goals>

            <goal> check </goal>

        </goals>

    </execution>

</executions>
```

```
</plugin>
```

The role of this configuration in Workshop 4 is twofold. Firstly, it enables the continuous analysis of the libraries used by DigiBank. Secondly, it directly prepares for future integration into GitHub Actions, since the same plugin can be relaunched in the pipeline with almost no changes. The `failBuildOnCVSS` threshold already introduces an automated safeguard, which is entirely consistent with the DevSecOps philosophy of the course.

2.8 Important Maven Commands for Dependency Analysis

Once the `pom.xml` file has been verified, several commands become important for the preparation phase of workshop 4. They are not yet used to draw conclusions about vulnerabilities, but rather to make the technical state of the project's dependencies visible and to ensure that the analysis will be performed on artifacts that are actually in use. The main commands are as follows:

```
mvn clean

mvn compile

mvn test

mvn clean verify

mvn dependency:tree

mvn org.owasp:dependency-check-maven:check

mvn org.owasp:dependency-check-maven:purge
```

Their role can be summarized as follows:

- **mvn clean** cleans up the previous build state.
- **mvn compile** verifies the compilation of the main code.
- **mvn test** executes existing tests.

- **mvn clean verify** executes the complete verification cycle, including plugins integrated into the verify phase.
- **mvn dependency:tree** allows you to visualize the direct and transitive dependencies actually embedded in the project.
- **mvn org.owasp:dependency-check-maven:check** explicitly launches the scan for known vulnerabilities in libraries.
- **mvn org.owasp:dependency-check-maven:purge** allows you to reset the local database of the tool if necessary.

The Dependency-Check report is usually generated in a directory such as `target/dependency-check-report.html`. At this stage, it is not yet necessary to interpret all the reported CVEs in detail; however, it is necessary to confirm that the report has been generated correctly and that the project is technically ready to be analyzed in a reproducible manner later in the workshop.

2.9 Preparing the DigiBank Dockerfile

One of the most important aspects of Workshop 4 is preparing a truly usable Dockerfile for analysis. A container security workshop is meaningless if the image isn't rebuildable, if the build file is outdated, or if it doesn't accurately reflect how DigiBank is actually packaged. The goal of this phase isn't yet to achieve complete hardening, but rather to have a clear, executable, and sufficiently readable containerization artifact to provide a solid foundation for subsequent risk analysis.

At the root of the project, it is recommended to create or revise a Dockerfile of this type:

```
FROM maven:3.9.8-eclipse-temurin-17 AS builder

WORKDIR /app

COPY pom.xml .

COPY common-module ./common-module

COPY customer-module ./customer-module

COPY account-module ./account-module
```

```
COPY transfer-module ./transfer-module

COPY digibank-web ./digibank-web


RUN mvn clean package -DskipTests


FROM eclipse-temurin:17-jre


WORKDIR /app

COPY --from=builder /app/digibank-web/target/digibank-web-*.jar
app.jar


EXPOSED 8080


ENTRYPOINT [ "java" , "-jar", "app.jar"]
```

This version is pedagogically valuable for several reasons. First, it clearly separates the build and execution phases using a multi-stage approach, directly aligning with the attack surface reduction best practices discussed in the course. Second, it avoids embedding Maven and source files in the final runtime image. Finally, it provides a simple yet realistic framework for observing what will subsequently be hardened: the base image, embedded content, exposed port, launch strategy, and the ability to run the application in a container.

It's important to note here that this Dockerfile isn't the ultimate goal for container security. Rather, it provides a foundation for analysis. Later in this workshop, we'll focus on

what needs improvement: choosing a more minimal image, using a non-privileged user, further reducing the number of components, better controlling copied files, and ensuring consistency with environment variables. But before hardening the system, we need a working prototype.

2.10 Preparing the .dockerignore file

A point often overlooked in containerization workshops, but particularly relevant here, concerns the `.dockerignore` file. In a DevSecOps approach, security doesn't depend solely on what the Dockerfile explicitly copies; it also depends on what the build context sends to Docker. If the repository contains reports, `.git` files, test exports, local secrets, unnecessary target directories, or environment files not intended for the image, their presence in the build context unnecessarily increases potential exposure.

It is therefore recommended to create a `.dockerignore` file of this type at the root of the project:

```
.git
.gitignore
.idea
.vscode
target
** /target
node_modules
dast
reports
* .log
.env
```

```
.env.*  
  
dependency-check-report.html
```

The purpose of this file is simple but important: to reduce noise, speed up the build process, and above all, prevent files not intended for the image from being sent to the Docker engine. This precaution is consistent with one of the major principles of session 4 of the course: reducing the attack surface also involves eliminating unnecessary files even before the final artifact is built.

2.11 Preparing a working docker-compose.yml

Even though container analysis can be performed using only the Dockerfile, it remains very useful, in a pedagogical context, to have a docker-compose.yml file that allows you to launch DigiBank and PostgreSQL in a consistent environment. Workshop 3 already demonstrated the value of Docker Compose for providing a reproducible environment. Here, this approach is applied not primarily for dynamic testing, but to prepare a realistic framework for analyzing containerized artifacts.

A starting file at the root of the project could be the following:

```
version: "3.9"  
  
services:  
  
  postgres:  
  
    picture: postgres:16  
  
    container_name: digibank-postgres  
  
    environment:  
  
      POSTGRES_DB: digibankdb  
  
      POSTGRES_USER: digibank
```

```
    POSTGRES_PASSWORD: digibank123

    ports:

      - "5432:5432"

    volumes:

      - postgres-data:/var/lib/postgresql/data

    healthcheck:

      test : [ "CMD-SHELL" , "pg_isready -U digibank -d digibankdb"]

      interval: 10s

      timeout: 5s

      retries: 5

  digibank:

    build: .

    container_name: digibank-app

    ports:

      - "8080:8080"

    environment:

      SPRING_PROFILES_ACTIVE: dev

      SPRING_DATASOURCE_URL: jdbc:postgresql://postgres:5432/digibankdb

      SPRING_DATASOURCE_USERNAME: digibank
```

```
    SPRING_DATASOURCE_PASSWORD: digibank123

    depends_on:

        postgres:

            condition: service_healthy

volumes:

    postgres-data:
```

This file has significant educational value in workshop 4. It demonstrates that containerization should not be considered in isolation, focusing solely on the Java binary, but rather in its relationship to the database, environment variables, and service startup order. It also naturally prepares the ground for the future CI/CD pipeline, where the same logic will need to be applied in an automated form.

2.12 Local image construction and first execution

Once the Dockerfile and docker-compose.yml are prepared, it's necessary to verify that the DigiBank image can actually be built. This step is essential because a container security workbench is worthless if the primary analysis medium—the image itself—doesn't exist or isn't working.

The basic commands are as follows:

```
docker build -t digibank:local .

docker images | grep digibank

docker run --rm -p 8080:8080 \

    -e SPRING_PROFILES_ACTIVE=dev \
```

```
SPRING_DATASOURCE_URL=jdbc:postgresql://host.docker.internal:5432/digibankdb \
-e SPRING_DATASOURCE_USERNAME=digibank \
-e SPRING_DATASOURCE_PASSWORD=digibank123 \
digibank:local
```

Or, in a more reproducible way:

```
docker compound up --build
```

The expected checks are as follows:

- The digibank:local image is well constructed.
- The container starts without a blocking error.
- The application is listening correctly on the expected port.
- The connection to PostgreSQL works if the associated environment is running.
- Sensitive variables are injected at runtime and not hard-coded into the image.

This step is not yet a security validation in the strict sense. Its primary purpose is to confirm that the containerized artifact about to be audited actually corresponds to a executable and consistent application.

2.13 Initial checks to be performed on the image and on the artifacts

Before even beginning a detailed vulnerability analysis, it is helpful to make a few simple observations about the resulting image. The goal is not to draw a conclusion yet, but to prepare the student for the types of questions they will need to explore further later. [Attachment]

The following commands are recommended:

```
docker picture inspect digibank:local

docker history digibank:local

docker run --rm digibank:local Java -version
```

These commands already allow us to observe several useful elements:

- The overall size of the image and the stacking of its layers.
- The possible presence of unnecessarily retained build steps.
- The base image actually used in the execution phase.
- The application launch strategy.
- Whether or not the image still relies on too broad an environment.

In a pedagogical context, it is important to emphasize that this initial observation is not a complete audit. It plays the same role as the initial mapping in DAST or the first instrumentation in SAST: it prepares the analysis by making visible the artifacts that will then be examined more rigorously.

2.14 Preparing workshop 4 working files in the repository

As in previous workshops, it is best to group the elements specific to this new phase into clear directories at the root of the project. This structuring has strong pedagogical value, as it transforms the results of the analysis into project artifacts rather than files scattered across the student's local machine.

A recommended organization might be the following:

```
digibank-parent/
├── .github/
│   └── workflows/
├── container-security/
│   └── reports/
```

```
|   └─ notes/
|   |   └─ container-review.md
|   └─ scripts/
├─ dependency-check-suppressions.xml
├─ Dockerfile
├─ .dockerignore
├─ docker-compose.yml
└─ pom.xml
```

The file `container-security/notes/container-review.md` may, for example, contain:

- The date of the analysis.
- The version of the project under study.
- The name of the constructed image.
- Initial observations on size, base and layers.
- The dependencies that appear to be the most sensitive according to the Maven or Dependency-Check report.
- Points to be addressed in the rest of the workshop.

This discipline directly prepares for the next parts, where you will be asked not only to launch tools, but also to interpret, prioritize and document the results in a traceable manner.

2.15 Preparing the future GitHub Actions workflow

Workshop 4 does not yet require finalizing the entire security CI/CD pipeline at this stage of Part 2, but it is useful to prepare the location and logic of the future workflow now. The course and previous workshops demonstrate that automation is not an optional add-on; it is the natural extension of manual and local controls.

Create the directory if it does not exist:

```
mkdir -p .github/workflows
```

Then prepare a working file that will be completed later in the workshop, for example:

```
.github/workflows/digibank-container-security.yml
```

The expected logic for this workflow is already known at this stage:

1. retrieve the source code;
2. prepare Java and Maven;
3. rebuild DigiBank;
4. run OWASP Dependency-Check;
5. build the Docker image;
6. publish useful reports and metadata as artifacts.

The key pedagogical point is this: GitHub Actions is not being prepared here as a simple technical integration tool, but as the future place where dependency and container controls will be systematically replayed. This idea is at the heart of the DevSecOps presented in the course.

2.16 Important commands of this phase

Once the environment is prepared, the essential commands for this section can be summarized:

```
CD digibank-parent  
  
mvn clean install  
  
mvn dependency:tree
```

```
mvn org.owasp:dependency-check-maven:check

docker build -t digibank:local .

docker image inspect digibank:local

docker history digibank:local

docker compose up --build

docker compose ps

docker compose logs -f
```

Each one has a specific function:

- **mvn clean install** rebuilds and validates the project.
- **mvn dependency:tree** makes the actual dependency graph visible.
- **mvn org.owasp:dependency-check-maven:check** prepares the analysis of known vulnerabilities.
- **docker build -t digibank:local .** produces the containerized image to be analyzed.
- **docker image inspect** and **docker history** allow an initial reading of the Docker artifact.
- **docker compose up --build** launches a reproducible environment combining the application and its database.

2.17 Final readiness check before detailed analysis

At the end of this phase, the student must be able to demonstrate that the environment for workshop 4 is truly ready. The minimum criteria are as follows:

- The DigiBank project opens correctly in IntelliJ IDEA.
- The complete Maven build runs without any blocking errors.
- The Maven parent contains the necessary configuration for dependency analysis.
- The OWASP Dependency-Check report can be generated.
- A consistent Dockerfile exists at the root of the project.
- A `.dockerignore` file properly limits the build context.
- The DigiBank Docker image can be built locally.

- A docker-compose.yml file allows the application to be launched with PostgreSQL in a consistent environment.
- Sensitive variables are injected at runtime and not stored in plain text in versioned artifacts.
- The location for the future GitHub Actions workflow is prepared.

A comprehensive verification sequence can therefore be given as follows:

```
CD digibank-parent

mvn clean install

mvn dependency:tree

mvn org.owasp:dependency-check-maven:check

docker build -t digibank:local .

docker compound up -d --build

docker compound ps

docker picture inspect digibank:local

docker history digibank:local
```

This sequence does not exhaust the security analysis, but it confirms that the field of observation is ready, that the artifacts really exist, and that the rest of the workshop can focus on concrete, reproducible and technically exploitable elements.

2.18 Expected result at the end of part 2

At the end of Part 2, the student should not yet have conducted a detailed vulnerability analysis of the dependencies or the DigiBank container, but they should have prepared all the necessary groundwork for this analysis. They have a rebuilt project, a Maven parent ready to expose and control dependencies, an analyzable Dockerfile, a consistent .dockerignore file, a working docker-compose.yml file, a locally rebuildable image, a generateable Dependency-Check report, and a clean repository structure to hold notes, scripts, and reports from Workshop 4.



In other words, DigiBank transitions from an application already built, tested, and observed in previous workshops to a technical artifact ready for a security analysis of its dependencies, images, and delivery chain. This transition is precisely what gives meaning to workshop 4 within the course's logic: after learning how to secure the code and then observe the application's behavior, it now becomes possible to examine, harden, and automate the security of what the application actually includes and carries within its deployment environment.

Part 3: Detailed Identification of Vulnerabilities Related to Dependencies and Containers in DigiBank

3.1 Role of this phase in the workshop

At this stage of Workshop 4, the student typically has a rebuilt DigiBank project, a Maven parent capable of exposing the dependency graph, a usable Dockerfile, a working Docker or Docker Compose environment, and an initial ability to generate reports on containerization libraries and artifacts. However, the pedagogical approach of this phase is not to mechanically copy the output of a scan or to assume that a list of CVEs is sufficient for a security analysis. Rather, it focuses on learning to read DigiBank as an expanded risk space, in which vulnerabilities arise not only from the business logic written by the team, but also from embedded libraries, base images, packaging choices, secrets potentially included in artifacts, and privileges anticipated by the container.



This third part therefore constitutes the intellectual core of Workshop 4. After preparing the working environment, the student must now examine the Maven dependencies, the parent pom.xml file, the application*.yml files, the Dockerfile, the .dockerignore file, the docker-compose.yml file, and the generated image in order to identify the main categories of weaknesses targeted by the course. The DigiBank project is particularly well-suited to this step, as it provides a realistic learning case: it works, compiles, runs, can be containerized and prepared for a pipeline, while still exposing technical decisions typical of projects in the process of being industrialized, and therefore not yet fully hardened.

Within the DevSecOps framework of the course, this phase has strategic importance. It serves as a reminder that software can be reasonably clean in terms of code and yet exposed to serious risks if it includes a vulnerable library, relies on an overly large base image, stores secrets in its build context, or prepares for an overly permissive execution. In other words, Workshop 4 teaches students that a system's security posture is no longer solely determined by Java classes or HTTP endpoints, but also by the entire software ecosystem actually used by the application during its assembly and execution.

3.2 Expected General Method

In a workshop focused on dependencies and containers, not all weaknesses appear in the same form. Some are directly visible in configuration files or the Dockerfile, others only become apparent when reading the Maven dependency graph, and still others only make sense when the constructed picture is compared to the general principles of attack surface reduction, provenance control, privilege minimization, and control of exposed secrets. The student must therefore learn to identify several complementary families of vulnerabilities, without limiting the analysis to a single type of tool or a single source of information.

The first category concerns vulnerable or outdated software dependencies. The course strongly emphasizes that a modern application executes a significant amount of code that it did not write itself, but rather imported via third-party libraries, frameworks, and components. A vulnerability in a direct or transitive dependency can therefore affect DigiBank without any obvious errors appearing in the local business logic. Identification then involves spotting outdated components, versions known to have public CVEs, and dependency chains that silently inherit these weaknesses.



The second category concerns weaknesses in the container's design itself. An image can be functional while remaining unnecessarily large, overly permissive, or too rich in unnecessary components. The course reminds us that the more packages, tools, and layers an image includes, the larger its potential attack surface. In DigiBank, this analysis must therefore focus on the nature of the base image, the distinction between the build and execution phases, the actual content of the final image, port exposure, and how the artifact prepares the service for execution.

The third category concerns the exposure of sensitive information within artifacts or their build context. The risk isn't simply that a password might still be present in a versioned configuration file; it's also the risk that a secret, a local file, a poorly managed variable, or an overly broad build context could end up being carried over into an image, copied into a Docker layer, or propagated through a CI/CD pipeline. This issue is central to Workshop 4, as the course explicitly emphasizes that secrets should not be embedded in images and that containers must remain separate from sensitive data that belongs to the runtime environment.

The fourth category concerns the privileges, permissions, and execution assumptions anticipated by the container. An application running as root, with overly broad permissions, or without preparation for a more restrictive execution mode, presents a higher level of exposure than a service designed with minimal privileges. Here again, the issue is not merely theoretical: if DigiBank were to be compromised at the application level, an overly permissive container would amplify the impact of that compromise. Identifying weaknesses therefore involves not only verifying that the service starts, but understanding the security conditions under which this startup is possible.

Finally, the fifth category concerns the temporal drift of artifacts. A dependency or image that is acceptable at the time of its creation can become problematic over time, as new public vulnerabilities are published. This dimension is pedagogically important because it shows that Workshop 4 does not simply aim for a snapshot of DigiBank, but rather the development of a continuous monitoring reflex. Students must understand that a security artifact is never permanently secure; it must be regularly reviewed, rebuilt, and reassessed.

3.3 Expected analysis method in DigiBank

As in the detailed analysis sections of previous workshops, students should not simply launch a tool and copy what they observe. A thorough analysis of dependencies and containers relies on a structured sequence. First, a cleanly rebuilt project must be started. Next, the files that actually control the packaging and import of third-party components must be examined. Only then does it become relevant to read the reports, observe the resulting picture, and then link the technical signals to clear risk categories.

The expected logic can be summarized as follows:

- review the parent pom.xml and any centralized versions to identify major dependencies and points of attention;
- produce the dependency graph to make visible the direct and transitive components actually embedded;
- launch OWASP Dependency-Check to identify known vulnerabilities reported in libraries;
- Review the Dockerfile, the .dockerignore file, and the docker-compose.yml file to understand the chosen packaging method;
- construct the image, observe its structure, its history, its size and its execution assumptions;
- link each observed signal or weakness to a principle of the course: origin, minimization, reduction of attack surface, control of secrets, reduction of privileges, continuous monitoring.

This method is important because it prevents two common errors. The first would be to assume that a dependency flagged by a tool automatically constitutes a proven exploit, whereas the course emphasizes the need for contextual interpretation. The second would be to assume that a container that starts is automatically a properly secured container. In both cases, the student must learn to move from the raw signal to a reasoned understanding of the risk.

3.4 Practical preparation of the DAST environment

One of the first areas of analysis in DigiBank concerns the Maven dependencies themselves. The Maven parent centralizes a significant portion of the library versions and choices, making it a crucial point of observation in Workshop 4. As the course reminds us, integrating a dependency is tantamount to integrating third-party code, with its

maintenance history, potential vulnerabilities, and sometimes transitive dependencies that are not immediately apparent. The application can therefore inherit significant weaknesses even if the local code is not the direct cause.

In DigiBank, several categories of weaknesses must be addressed at this level. The first is the use of outdated or poorly maintained versions of core frameworks. When versions locked in the parent pom.xml file are no longer aligned with recent security patches, the application carries a security debt at the very heart of its technical foundation. This situation is particularly critical for core components such as Spring Boot, API exposure libraries, and certain cross-dependencies used across multiple modules.

The second category concerns vulnerable transitive dependencies. In modern projects, the components actually executed are not limited to the libraries explicitly declared by the developer. A direct dependency can introduce several others, which themselves contain still more. DigiBank is no exception to this pattern, and this is precisely why the course emphasizes the need to map the entire software graph and not just skim the pom.xml file. A significant vulnerability can thus stem from an artifact never directly mentioned in the project's code.

The third category concerns unnecessary, redundant, or imported dependencies, whether imported out of habit rather than necessity. An unused or poorly justified dependency nonetheless adds code, sometimes loaded classes, sometimes additional transitive components, and therefore potentially additional CVEs. In the interest of reducing the attack surface, students must learn to consider that an unnecessary library is not simply a matter of technical cleanliness: it is an unjustified increase in the software surface area that needs to be secured.

The fourth category concerns dependencies whose impact must be contextualized. The course rightly points out that identifying a known vulnerability does not automatically mean that it is exploitable under all conditions. In DigiBank, some libraries flagged by OWASP Dependency-Check may require a more detailed analysis: Is the functionality in question actually used? Is the component directly exposed? Is the vulnerable version used for marginal purposes or is it central to the application? This interpretive approach is essential, because the goal of Workshop 4 is not to produce a raw list of alerts, but a prioritized assessment of the risk.

To support this analysis, the most useful mvn commands are as follows:

```
mvn dependency:tree
```

```
mvn org.owasp:dependency-check-maven:check  
  
mvn clean verify
```

The expected outcome is not simply an HTML report. The student must be able to identify, within this report and the Maven graph, which dependencies are most critical, which are direct or transitive, which can be reasonably updated, and which require further documentation or scrutiny before a decision is made. This analytical ability then prepares the ground for remediation.

3.5 Vulnerabilities searched in the Dockerfile and in the DigiBank image

Beyond Maven dependencies, Workshop 4 requires viewing DigiBank as a containerized project. The Dockerfile should therefore not be considered a simple technical file intended to "make Docker work," but rather a security document in its own right. It specifies the choice of base image, whether or not the build and runtime phases are separated, the files actually copied into the final image, the anticipated runtime environment, and, indirectly, the software surface embedded within the container.

The first potential vulnerability to look for here concerns the origin and nature of the base image. The course emphasizes that an image downloaded from a public registry is not automatically secure simply because it is available, popular, or easy to use. Therefore, in DigiBank, students must consider whether the chosen base image is reasonably maintained, suitable for its intended use, and compatible with a minimization effort. A base image that is too large or too general carries more components, and thus a larger potential attack surface.

The second vulnerability concerns the size and content of the final image. If the Dockerfile doesn't properly distinguish between the build and execution phases, it can include compilation tools, source code, intermediate artifacts, or files in the final image that serve no purpose in the actual DigiBank runtime. This point is central to the course, which presents minimization as a fundamental principle of container security. A lighter image is not only more efficient; it is also more readable, easier to monitor, and less susceptible to the exploitation of unnecessary components.

The third vulnerability concerns excessive file copying within the build context or the image itself. A Dockerfile can be functional while still copying the repository too extensively, including reports, local files, technical exports, unnecessary metadata, or configurations not intended for runtime. This problem is particularly significant because it can introduce elements into the Docker layer that should never have left the development machine or the Git repository. Therefore, students must carefully read the COPY instructions, the actual Docker context being used, and check for the possible existence of an insufficient .dockerignore file.

The fourth vulnerability concerns port exposure, launch strategy, and execution assumptions. Exposing a port is not inherently a vulnerability, but it must correspond to a real and well-defined need. Similarly, launching the application with very broad permission assumptions or without preparing for a more restrictive execution is a sign of weakness. The course reminds us here that the container already prepares a behavioral model; it doesn't simply passively wrap the service.

The fifth vulnerability concerns the readability and security maintainability of the Dockerfile itself. A file that is too opaque, too complex, or poorly designed makes future hardening, security reviews, and clean integration into a CI/CD pipeline more difficult. Workshop 4 focuses not only on the image as a final product, but also on the fact that this image must be able to be rebuilt, reviewed, and maintained over time. Therefore, lasting security also requires a minimum level of quality in the packaging structure.

For this part of the analysis, useful commands include, for example:

```
docker build -t digibank:local .  
  
docker picture inspect digibank:local  
  
docker history digibank:local
```

The goal is not just to confirm that the image exists, but to link what is observed to concrete security questions: what base image is used, what layers are created, what is the overall size of the artifact, what instructions seem to copy many elements, and what execution the container is actually preparing.

3.6 Vulnerabilities related to secrets and embedded configuration

One of the most important weaknesses to look for in DigiBank at this stage concerns the presence of sensitive information in versioned files, the build context, or the container image. The course explicitly addresses this point, reminding us that a container should not contain passwords, keys, tokens, or confidential configuration files. An image circulates between environments, registries, and actors; any secrets it carries are therefore likely to be disseminated much more widely than intended.

In DigiBank, the first area to examine remains the Spring configuration files. Even if previous workshops have already introduced the logic of environment variables, students must verify that this discipline is indeed followed and that no sensitive identifiers remain hard-coded in an `application-dev.yml`, a `docker-compose.yml`, a sample command, a local script, or an export left in the repository. The vulnerability here is not only technical; it is also pedagogical. A bad practice retained in one sample project tends to become a pattern replicated elsewhere.

The second area concerns the Docker context itself. Even if a secret isn't intentionally copied into the final image, it can still be included in the build context if the `.dockerignore` file is missing or poorly designed. This means that an `.env` file, a report, a collection export, internal notes, or other local files can be sent to the Docker engine and then potentially copied into a layer according to the Dockerfile's logic. This situation clearly illustrates how a seemingly peripheral weakness can become a risk of spreading throughout the software build chain.

The third area concerns the environment variables themselves when they are poorly managed. Using environment variables is a significant improvement over hard-coded secrets, but it doesn't absolve the student of the need for security considerations. It's still necessary to verify that sensitive values aren't committed to a versioned local file, that the examples don't normalize bad practices, and that the image build doesn't persistently embed these variables in the Docker layers. Here again, the challenge isn't just to "make DigiBank work," but to ensure that the way it works remains compatible with a credible security chain.

This part of the analysis can usefully be supported by a cross-review of:

- application.yml and application-dev.yml;
- docker-compose.yml;
- .dockerignore;
- Dockerfile;
- the launch commands proposed in the working documentation.

The expected outcome is that the student will be able to explain where a secret could still leak, how it could become entangled in an artifact, and why this exposure greatly increases the risk once the image is copied, stored or shared.

3.7 Vulnerabilities related to container privileges and attack surface

Another significant vulnerability in DigiBank concerns container privileges and attack surface. The course emphasizes that a container should not run the application with more privileges than necessary, and that an image must be designed to limit opportunities for abuse, exploration, or privilege escalation. This issue is not simply an abstract "production" best practice; it directly impacts how the artifact is built from the earliest iterations of the project.

The first typical weakness to look for is implicit execution as root. Many images run in this mode by default, which simplifies startup but creates a much more permissive environment than it should be. In DigiBank, even though the project remains educational, students must learn to consider this situation a genuine sign of weakness. Compromising the application in an overly permissive container opens up a more favorable environment for an attacker than running it as an unprivileged user.

The second weakness concerns broad permissions on the container's files or working system. If the application can write too freely, modify areas that don't need to be modified, or run without a clear separation between the application artifact and the environment, future hardening becomes more difficult. The course rightly emphasizes that a secure container is not simply an application package; it is a component designed to reduce the potential for abuse and facilitate control mechanisms.

The third weakness concerns unnecessary components that are still present in the runtime image. An excessively large image may contain shells, diagnostic tools, additional libraries, or other packages that increase the attack surface without being required by the DigiBank service. This situation must be addressed in Workshop 4, as it concretely illustrates the attack surface reduction principle studied in the course.



The fourth weakness concerns the implicit dependence on a very permissive environment. A container that only functions correctly because it benefits from broad privileges, ubiquitous files, widely exposed ports, or a very comfortable context is a container that is more difficult to secure in the long term. Students must therefore learn to identify not only explicit weaknesses, but also the assumptions of convenience that mask poor design restraint.

3.8 Use of curl for some targeted tests

As in previous workshops, detailed identification doesn't end with the mere appearance of results. It requires reasoned interpretation. OWASP Dependency-Check, for example, might flag vulnerabilities in libraries that don't all have the same impact on DigiBank. Similarly, observing a somewhat large image or a container launched as root doesn't necessarily mean the application is already compromised. What the student needs to learn here is how to transform technical signals into a structured risk assessment.

This interpretation must answer several questions. Is the vulnerable dependency direct or transitive? Is it used in a sensitive functional area? Is the reported vulnerability still active in the actual embedded version? Does the Dockerfile prepare for an overly permissive execution out of habit, or is there a real constraint that explains it? Does the configuration file still contain secrets, or has the risk been moved elsewhere in the build chain? Is the image unnecessarily broad simply through negligence, or because the distinction between build and runtime has not yet been made?

In an educational context, it is legitimate to recommend a structured presentation, for example in the form of a fact sheet for each vulnerability or a table with the following headings:

- component concerned;
- file or observed artifact;
- type of weakness;
- tool or means of identification;
- estimated criticality level;
- probable impact on DigiBank;
- principle of the course in question;
- remediation approach envisaged.



This discipline directly prepares students for the following section, which focuses on detailed remediation. It also prevents students from addressing all alerts at the same level, which would contradict the DevSecOps logic of the course, based on prioritization, contextual understanding, and continuous integration of controls.

3.9 Expected feedback sheet for each dynamic vulnerability

At the end of this third part, the student should be able to navigate DigiBank not just as a Java application to compile or run, but as a collection of artifacts to audit from the perspective of dependencies and containerization. They should be able to recognize patterns that indicate a potentially vulnerable library, a transitive dependency to monitor, an overly large base image, insufficient separation between build and runtime, imperfect secret management, an unnecessarily large attack surface, or an overly permissive container execution.

It must also be possible to justify why these points constitute vulnerabilities, or at least significant weaknesses, within a DevSecOps framework. In other words, it's no longer just about identifying what could be "improved," but about understanding what expands DigiBank's actual risk surface, what weakens its delivery chain, and why early remediation is preferable to late remediation after the artifact has been deployed in a pipeline or across multiple environments. This understanding directly prepares the next phase, where the goal will be to transform this detailed identification into structured remediation work on dependencies, the Dockerfile, secrets, privileges, and future CI/CD logic.



Part 4: Detailed Remediation of Dependency and Container Vulnerabilities in DigiBank

4.1 Role of this phase

At this stage of the workshop, DigiBank should no longer be viewed simply as a functional application, but as a complete software artifact, composed of code, dependencies, container images, configuration files, and pipeline elements that can introduce their own vulnerabilities. The course specifically emphasizes that a significant portion of modern risk no longer stems solely from locally written business logic, but also from what is imported, packaged, exposed, and executed within the software delivery environment.

In this logic, remediation does not consist of simply “updating a few versions” or “making Docker work”, but of progressively transforming DigiBank into a more understated, more explicit, more controlled and more consistent application with the principles of attack surface reduction and hardening presented in session 4. As in TP2, each

weakness is therefore treated here according to a stable structure: initial code or configuration, corrected version, explanation, verification commands and expected impact.

4.2 Remediation of an overly large and overexposed Docker image

In an initial educational version, it is common for a Java application to be containerized with a generic, sometimes complete, image, and for the final image to include unnecessary build tools or components. However, the course emphasizes that reducing the attack surface begins with minimizing the embedded components and separating the build and execution phases. [Attachment]

Vulnerable code or initial configuration

Dockerfile

```
FROM maven:3.9.8-eclipse-temurin-17

WORKDIR /app

COPY . .

RUN mvn clean package -DskipTests

EXPOSED 8080

CMD ["java", "-jar", "digibank-web/target/digibank-web-1.0.0-SNAPSHOT.jar"]
```

In this version, the final image contains Maven, the entire source code, build files, and potentially elements that have no runtime function. This approach works technically, but it unnecessarily increases the image size, the number of components present, and therefore the potential attack surface. [Attachment]

Code corrected

Dockerfile

```
FROM maven:3.9.8-eclipse-temurin-17 AS builder

WORKDIR /build

COPY pom.xml .

COPY common-module ./common-module

COPY customer-module ./customer-module

COPY account-module ./account-module

COPY transfer-module ./transfer-module

COPY digibank-web ./digibank-web

RUN mvn clean package -DskipTests

FROM eclipse-temurin:17-jre-alpine

WORKDIR /app
```

```
COPY --from=builder
/build/digibank-web/target/digibank-web-1.0.0-SNAPSHOT.jar
app.jar

EXPOSED 8080

ENTRYPOINT [ "java" , "-jar", "app.jar"]
```

Explanation

The fix introduces a multi-stage build, allowing Maven to be used only in the build stage, and then retaining only the Java runtime and the strictly necessary executable artifact in the final image. This change directly illustrates the best practice of reducing the attack surface described in the course: the fewer components the image contains, the more readable and lightweight it is, and the less exploitable material it contains in case of a vulnerability.

It also improves the separation of responsibilities between artifact production and service execution. The final image no longer requires the source code or compilation tools, which corresponds to a basic but essential hardening principle in a DevSecOps approach applied to containers.

Verification commands

```
docker build -t digibank:secure .

docker images | grep digibank

docker history digibank:secure

docker run --rm -p 8080:8080 digibank:secure
```

Expected impact

The first expected impact is a reduction in image size and the number of layers required for execution. The second is a decrease in the attack surface due to the presence of

unnecessary build tools in the final container. This improvement aligns perfectly with the container minimization and hardening principles presented in session 4.

4.3 Remediation of container execution as root

Another common weakness of a beginner's container is running the application with the default user, which is often root. The course, however, emphasizes privilege limitation as a fundamental principle of hardening, because a functional but overly permissive container remains easier to hijack or exploit in the event of an incident.

Vulnerable code or initial configuration

Dockerfile

```
FROM eclipse-temurin:17-jre-alpine

WORKDIR /app

COPY target/digibank-web-1.0.0-SNAPSHOT.jar app.jar

EXPOSE 8080

ENTRYPOINT [ "java" , "-jar", "app.jar"]
```

Explanation

The fix creates a dedicated, non-privileged user, assigns the necessary permissions to the application directory, and then forces the Java process to run under that user. This hardening measure reduces the potential impact of unintended behavior within the

container, since an application compromise no longer occurs immediately with the highest privileges.

This practice does not make a container "invulnerable," but it perfectly illustrates the course's idea that security also involves limiting the potential effects of an existing weakness. In other words, hardening does not replace patching vulnerabilities, but it reduces their potential impact.

Verification commands

```
docker build -t digibank:nonroot .  
  
docker run --rm digibank:nonroot id  
  
docker run --rm -p 8080:8080 digibank:nonroot
```

Expected impact

The container should no longer run under root, but under the dedicated user defined in the image. This improvement strengthens the artifact's hardening level and aligns with the session privilege reduction recommendations of Session 4.

4.4 Remediation of secrets exposed in the containerized configuration

Lab 2 already demonstrated that the presence of secrets in plain text within the configuration constitutes a significant weakness. In the context of containers, this risk becomes even more critical, as the files embedded in the image can be distributed, inspected, or reused in other environments, transforming a simple pedagogical convenience into a persistent bad practice.

Vulnerable code or initial configuration

application-dev.yml file

```
spring:  
  
  datasource:  
  
    URL: jdbc:postgresql://localhost:5432/digibankdb
```

```
username: digibank

password: digibank123
```

Dockerfile

```
ENV SPRING_DATASOURCE_USERNAME=digibank

ENV SPRING_DATASOURCE_PASSWORD=digibank123
```

In this version, identifiers are visible either in a versioned file or directly in the image via hard-coded variables. However, the course explicitly classifies exposed secrets among recurring weaknesses to detect and correct in a DevSecOps pipeline.

Code corrected

application.yml file

```
spring:

  datasource:

    url: ${ SPRING_DATASOURCE_URL : jdbc : postgresql ://
localhost : 5432 / digibankdb }

    username: ${ SPRING_DATASOURCE_USERNAME }

    password: ${ SPRING_DATASOURCE_PASSWORD }
```

Dockerfile

```
FROM eclipse-temurin:17-jre-alpine
```

```
WORKDIR /app

COPY target/digibank-web-1.0.0-SNAPSHOT.jar app.jar

EXPOSED 8080

ENTRYPOINT [ "java" , "-jar", "app.jar"]
```

Execution :

```
docker run --rm -p 8080:8080 \

-e

SPRING_DATASOURCE_URL=jdbc:postgresql://host.docker.internal:5432/digibankdb \

-e SPRING_DATASOURCE_USERNAME=digibank \

-e SPRING_DATASOURCE_PASSWORD='strongpassword' \

digibank:secure
```

Explanation

The fix involves removing secrets from the repository and image, then externalizing them via runtime-injected environment variables. This practice is consistent both with Lab 2, which condemned secrets visible in the configuration, and with the course's DevSecOps logic, which states that sensitive information should not be hard-coded into build or deployment artifacts.

It also naturally prepares for integration into a CI/CD pipeline, where these values can be injected via GitHub Actions secrets or equivalent mechanisms. The goal is not simply

to "hide a password," but to shift the responsibility for secret management to the appropriate level of the system.

Verification commands

```
grep -R "digibank123" .

docker history digibank:secure

docker run --rm \

  -e SPRING_DATASOURCE_USERNAME=digibank \

  -e SPRING_DATASOURCE_PASSWORD=test \

  digibank:secure
```

Expected impact

Secrets should no longer appear in versioned files or in the Docker image history. This fix reduces the risk of credential leaks and aligns DigiBank with the best practices for secret management outlined in the lecture and Lab 2.

4.5 Remediation of insufficiently enforced business rules in transfers

The course reminds us that an application can be well-written yet still be exposed due to a vulnerable third-party library, whether direct or transitive. Dependency analysis is therefore not a secondary task: it is an integral part of a security posture, as it focuses on what the project actually includes at the time of build and deployment.

Vulnerable code or initial configuration

Example in pom.xml

```
< properties >

  < spring.boot.version > 3.3.2 </ spring.boot.version >

  < springdoc.version > 2.6.0 </ springdoc.version >
```

```

</ properties >

< dependency >

    < groupId > org.springdoc </ groupId >

    < artifactId > springdoc-openapi-starter-webmvc-ui </
artifactId >

    < version > ${springdoc.version} </ version >

</dependency>

```

Or, in a more general case, a transitive dependency flagged by OWASP Dependency-Check can appear in the Maven graph without being explicitly controlled. Lab 2 already showed that such a situation must be addressed either by updating, targeted exclusion, or component replacement.

Code corrected

```

< properties >

    < spring.boot.version > 3.4.3 </ spring.boot.version >

    < springdoc.version > 2.8.9 </ springdoc.version >

</ properties >

< dependency >

    < groupId > org.springdoc </ groupId >

        < artifactId > springdoc-openapi-starter-webmvc-ui </
artifactId >

    < version > ${springdoc.version} </ version >

```

```
</dependency>
```

Or, if the vulnerability stems from a transitive verb:

```
< dependency >

  < groupId > org.example </ groupId >

  < artifactId > example-artifact </ artifactId >

  < version > 1.2.3 </ version >

  < exclusions >

    < exclusion >

      < groupId > vulnerable.group </ groupId >

      < artifactId > vulnerable-artifact </ artifactId >

    </exclusion>

  </ exclusions >

</dependency>

< dependency >

  < groupId > vulnerable.group </ groupId >

  < artifactId > vulnerable-artifact </ artifactId >

  < version > fixed-version </ version >

</dependency>
```

Explanation

The fix should be guided by the scan report and not by blindly updating the entire project. Assignment 2 already emphasized this point: you must identify the actual dependency involved, understand whether it is direct or transitive, and then choose a remediation strategy compatible with DigiBank's architecture.

This approach aligns perfectly with session 4, which emphasizes that identifying a known vulnerability in a library is a first step, but it must be followed by context analysis and reasoned remediation. A third-party dependency is not neutral: it is part of the system's actual attack surface.

Verification commands

```
mvn dependency:tree

mvn org.owasp:dependency-check-maven:check
-DnvdApiKey=YOUR_NVD_API_KEY

mvn clean verify
```

Expected impact

The dependency in question should no longer appear as vulnerable in the report or should present a reduced and justified level of risk. This correction directly improves the project's posture with regard to known vulnerabilities in third-party components, in accordance with the dependency analysis logic of the course.

4.6 Remediation of excessive Swagger exposure in a containerized environment

Lab 2 reminded us that Swagger and OpenAPI are very useful pedagogically, but that they also increase the exposure level when left unrestricted in more sensitive contexts. Session 4 specifically encourages us to think not only in terms of functionality, but also in terms of minimizing visible interfaces and components.

Vulnerable code or initial configuration

application.yml file

```
springdoc:

  api-docs:

    enabled: true

  swagger-ui:

    enabled: true
```

This configuration makes interactive documentation permanently available, including in a containerized deployment that might be exposed beyond the strictly local environment. While this aids testing, it also simplifies API mapping. [Attachment]

Code corrected

application.yml file

```
springdoc:

  api-docs:

    enabled: false

  swagger-ui:

    enabled: false
```

application-dev.yml file

```
springdoc:

  api-docs:

    enabled: true

  swagger-ui:

    enabled: true
```

Execution :

```
docker run --rm -p 8080:8080 -e SPRING_PROFILES_ACTIVE=dev
digibank:secure
```

or, in a standard environment, without a developer profile:

```
docker run --rm -p 8080:8080 digibank:secure
```

Explanation

The solution is to limit Swagger activation to the development profile and disable documentation by default in other contexts. This strategy is consistent with the logic of minimizing the attack surface: a convenient interface for development should not become a habit of permanent exposure.

It also shows that hardening isn't just about "blocking," but about explicitly associating each feature with a legitimate use case. In this sense, DigiBank becomes cleaner functionally and more consistent in terms of security exposure.

Verification commands

```
docker run --rm -p 8080:8080 digibank:secure

curl http://localhost:8080/swagger-ui.html

curl http://localhost:8080/api-docs

docker run --rm -p 8080:8080 -e SPRING_PROFILES_ACTIVE=dev
digibank:secure

curl http://localhost:8080/swagger-ui.html
```

Expected impact

Swagger should no longer be accessible in the standard profile, but only in the development profile. This fix reduces the application's visible area and aligns with the recommendation in Lab 2 to tailor interface exposure to different environments.

4.7 Remediation of the lack of automated dependency and image checking in the pipeline

The course emphasizes that dependency and container security should not remain a one-off or manual activity. A useful local fix must be integrated into the CI/CD pipeline so that control is reproducible, traceable, and applied to every significant project change. [Attachment]

Vulnerable code or initial configuration

Absence of workflow, or workflow limited to the build:

```
name: DigiBank Build

on:
  push:
    branches: [ main ]

jobs :
  build:
    runs-on: ubuntu-latest
    steps:
```

```
- uses: actions/checkout@v4

- uses: actions/setup-java@v4

  with:

    distribution: temurin

    java-version: 17

- run: mvn clean verify
```

In this version, the project compiles, but neither the dependencies nor the containerized artifact are integrated into a continuous monitoring system. Yet the course presents the pipeline as one of the central areas for integrating security within a DevSecOps approach.

Code corrected

File .github/workflows/digibank-container-security.yml

```
name: DigiBank Container and Dependency Security Pipeline

on:

  push:

    branches: [ hand, master, develop ]

  pull_request:

    branches: [ hand, master, develop ]

jobs :

  build-scan-container:

    runs-on: ubuntu-latest
```

```
steps:

  - name: Checkout repository

    uses: actions/checkout@v4


  - name: Set up JDK 17

    uses: actions/setup-java@v4

    with:

      distribution: temurin

      java-version: 17

      hidden: maven


  - name: Build and verify DigiBank

    run: mvn clean verify


  - name: Run OWASP Dependency-Check

    run: mvn org.owasp:dependency-check-maven:check

    approx:

      NVD_API_KEY: ${ secrets . NVD_API_KEY }


  - name: Build Docker picture
```

```
run: docker build -t digibank:ci .

- name: Export Docker picture metadata

  run: docker picture inspect digibank:ci >
docker-image-inspect.json

- name: Upload Dependency-Check report

  uses: actions/upload-artifact@v4

  with:

    name: dependency-check-report

    path: target/dependency-check-report.html

- name: Upload Docker metadata

  uses: actions/upload-artifact@v4

  with:

    name: docker-image-metadata

    path: docker-image-inspect.json
```

Explanation

The solution involves integrating the build, dependency scanning, and Docker image construction into a unified GitHub Actions workflow. This approach directly aligns with the

positioning of security within the CI/CD pipeline as described in the course: security is no longer a separate control, but a normal component of the software development chain.

It also allows remediation to be observed over time. A good DevSecOps practice is only truly valuable if it can be replayed with each significant change, and if its results can be reviewed, archived, and compared across multiple pipeline executions.

Verification commands

```
git add .github/workflows/digibank-container-security.yml
git commit -m "Add container and dependency security pipeline"
git push origin hand
```

Then check in GitHub Actions:

- Job completed correctly.
- Presence of the Dependency-Check report.
- Presence of metadata for the constructed image.

Expected impact

DigiBank has an initial pipeline capable of rebuilding the application, controlling its dependencies, and producing a verifiable Docker artifact. This fix improves security continuity and concretely translates the integration of Session 4 into a real-world CI/CD practice.

4.8 Remediation of a lack of verification discipline after correction

As in Lab 2, remediation cannot be considered complete unless it is followed by explicit technical revalidation. The course also reminds us that practices of updating, reducing the attack surface, and hardening are more effective when they become regular, integrated, and repeatable mechanisms.

Vulnerable code or initial configuration

There's no single syntactic weakness here, but rather a procedural one: the fixes are applied locally without systematic re-reading of the image, without rerunning dependency scans,

and without global consistency checks after modification. This situation leaves the project in a precarious position, because an unvalidated "good security intention" remains an incomplete fix.

Corrected version

Standardized verification sequence:

```
mvn clean

mvn test

mvn clean verify

mvn dependency:tree

mvn org.owasp:dependency-check-maven:check
-DnvdApiKey=YOUR_NVD_API_KEY

docker build -t digibank:final .

docker run --rm -p 8080:8080 digibank:final

docker history digibank:final
```

If GitHub Actions is enabled:

```
git push origin develop
```

Explanation

The solution here involves institutionalizing a revalidation sequence after each significant remediation. This practice aligns perfectly with the DevSecOps logic of Lab 2: a fix is only valuable if it is understood, applied correctly, and then verified reproducibly on the actual project.



In the context of dependencies and containers, this revalidation must cover at least three dimensions: build stability, the state of dependencies, and the consistency of the containerized artifact. Without this, the project risks reverting to a declarative security approach, where best practices are stated but not truly consolidated in technical usage.

Verification commands

The commands above constitute precisely the expected verification. They must be executed after modifications to the Dockerfile, pom.xml, Spring configuration files, or CI/CD workflows.

Expected impact

The corrected project must not only appear safer to read, but also demonstrate that it remains compilable, executable, has a cleaner Docker image, and is better managed in terms of its dependencies. It is this capacity for structured revalidation that transforms remediation into a credible continuous security approach.

Expected result

Upon completion of Part 4, the student should no longer consider dependencies and containers as mere technical details surrounding DigiBank, but as central components of their security posture. They should be able to explain why an overly large image increases the attack surface, why a container should not run as root, why secrets should not be embedded in the image or repository, why a vulnerable dependency weakens the system even if the local code appears clean, and why these controls must be integrated into the CI/CD pipeline.

In practical terms, the corrected version of DigiBank should appear lighter, more explicit, better compartmentalized, and more easily verifiable. This progression is perfectly consistent with session 4 of the course, which demonstrates that modern security is not just about verifying what is written in the source code, but also about controlling what is imported, packaged, exposed, and executed in the software delivery chain.

Would you like me to now transform this Part 4 into a version that can be directly pasted into your final document, with even more academic numbering and wording consistent with your other sections?



Part 5: Implementing the Security CI/CD Pipeline

5.1 Role of this phase in the workshop

After identifying vulnerabilities related to dependencies and containers, and after initial local remediation on DigiBank, the next step is to industrialize these controls within a security CI/CD pipeline. This phase aligns with the overall logic of the course, which presents the pipeline not as a simple build mechanism, but as a central space where security must be continuously, automatically, and observably integrated.

In previous workshops, DigiBank was already structured as a multi-module Maven project, instrumented with monitoring tools, and then enhanced with GitHub Actions

workflows to automate certain checks. Part 5 directly extends this progression: it's no longer just about executing commands locally, but about building a pipeline capable of automatically replaying the essential checks with each update to the repository.

From a pedagogical standpoint, this phase is important because it shifts the student from a focus on one-off fixes to a focus on continuous security. A dependency fixed today can become problematic again tomorrow if a new, vulnerable version is reintroduced, if the Dockerfile deviates, or if a configuration change weakens the application; the pipeline serves precisely to detect these regressions as early as possible.

The goal is therefore not to build a complete industrial DevSecOps factory at this stage, but to implement a credible core of automated controls. In DigiBank, this core must at a minimum allow the project to be rebuilt, tests to be run, dependency analysis to be rerun, the Docker image to be produced, and usable proof artifacts to be published in GitHub Actions.

5.2 Technical prerequisites before CI/CD integration

Before creating the pipeline, the student must verify that DigiBank remains stable in their local environment. Previous workshops have already emphasized the importance of a consistent system, with Java 17, Maven, Git, and Docker correctly installed, because a pipeline should not be used to mask a locally inconsistent project. [Attachment]

A first local verification sequence can be presented as follows:

```
Java -version
mvn -version
git --version
docker --version
docker compound version
```

If any of these commands fail, the environment must be corrected before proceeding, as a reproducible CI/CD pipeline relies on the stability of the underlying tools.

Lab 2 specifically reminds us that false version or tooling issues can obscure the interpretation of security results and lead to overlooking the project's true vulnerabilities.

Once the environment is validated, you must then return to the root of DigiBank's Maven parent project and rebuild the application cleanly:

```
CD digibank-parent  
mvn clean verify
```

This command remains the natural entry point for ensuring the overall consistency of the project. Following on from Lab 2, it allows us to confirm that the Maven modules remain properly aligned, that dependencies are resolved, that tests run correctly, and that the technical foundation is stable enough to be deployed in GitHub Actions.

It is also recommended to perform a clean Git backup before adding or modifying the CI/CD pipeline. This practice aligns with the traceability principles already emphasized in previous workshops:

```
git status  
git add .  
git commit -m "Baseline before CI/CD security pipeline"
```

This step makes it possible to clearly distinguish the state of the project before CI/CD instrumentation and the state enriched by the workflow files, any Maven adjustments and associated security scripts.

5.3 Expected technical checks

Within DigiBank, the security CI/CD pipeline can be organized around five complementary responsibilities. The first is to retrieve the source code and prepare the Java/Maven environment consistent with the project; the second is to execute the build and tests; the third is to run dependency scans; the fourth is to build the Docker image; and the fifth is to publish useful reports and metadata as artifacts.

This structure is consistent with how the course positions security in the CI/CD pipeline. Controls should not be concentrated solely at the end of the chain, but placed where they are most useful: dependencies at build time, artifacts at packaging time, and traceability at the time of results publication.

It is also important to maintain a progressive approach. Lab 2 already shows that a simple but well-understood pipeline is better than an accumulation of poorly mastered tools; it is therefore preferable, initially, to properly automate Maven, OWASP Dependency-Check and Docker build, before possibly adding other more advanced scanners.

In this context, the recommended location for the workflow is:

```
.github/workflows/digibank-security-pipeline.yml
```

This file will be the central point of automation for Part 5. It must be readable enough for a student to understand not only what they are doing, but also why each step contributes to the overall security posture of the project.

5.4 Expected technical checks with Dependency-Check and Trivy

Final validation must rely on a clear sequence of technical checks. The project must first be rebuilt cleanly from its Maven root, then the unit tests must be rerun to ensure that the remediations have not broken the core business rules. Next, the security tools must be rerun to measure the project's status after the fixes.

A consistent local verification sequence can be presented as follows:

```
mvn clean

mvn test

mvn clean verify

mvn                    org.owasp:dependency-check-maven:check
-DnvdApiKey=YOUR_NVD_API_KEY
```

```
docker build -t digibank:secure .  
  
trivy picture --severity HIGH, CRITICAL --exit-code 1  
digibank:secure
```

In this sequence, Maven checks the overall project consistency, OWASP Dependency-Check confirms the status of Java dependencies, and then Docker builds the containerized DigiBank artifact. Trivy then intervenes on the generated image to detect critical vulnerabilities present in the delivery artifact itself, usefully complementing the analysis of Maven dependencies alone.

Depending on the level of detail desired in the practical exercise, it is also possible to generate a usable Trivy report as a file. A command like the following, for example, allows you to keep a structured record of the scan:

```
trivy picture --format json --output trivy-report.json  
digibank:secure
```

This preservation of results is important for knowledge management. The Dependency-Check report and the Trivy report do not serve exactly the same purpose, but comparing them allows for a better understanding of the difference between a vulnerability inherited from a Java library and a vulnerability present in the final, executable image. This distinction directly aligns with the course's focus, which progressively expands the security of code to include dependencies, containers, and the CI/CD pipeline.

5.5 Creating the GitHub workflow Main actions

Create or open the following file:

Creating the main GitHub Actions workflow

```
.github/workflows/digibank-security-pipeline.yml
```



Building on previous sections, final validation becomes more relevant when automated within GitHub Actions. The course presents the CI/CD pipeline as a strategic space for integrating security controls, rather than simply as a compilation and testing mechanism. Adding Trivy to the workflow makes image verification reproducible, visible, and consistent with the DevSecOps principles being explored.

An example of a GitHub Actions job integrating Maven, Dependency-Check and Trivy can be formulated as follows:

```
name: DigiBank Security Validation Pipeline
```

```
on:
```

```
  push:
```

```
    branches: [ hand, master, develop ]
```

```
  pull_request:
```

```
    branches: [ hand, master, develop ]
```

```
jobs :
```

```
  security-validation:
```

```
    runs-on: ubuntu-latest
```

```
    steps:
```

```
      - name: Checkout repository
```

```
        uses: actions/checkout@v4
```

```
- name: Set up JDK 17

uses: actions/setup-java@v4

with:

  distribution: temurin

  java-version: 17

  hidden: maven

- name: Build and test DigiBank

run: mvn clean verify

- name: Run OWASP Dependency-Check

run: mvn org.owasp:dependency-check-maven:check

approx:

  NVD_API_KEY: ${ secrets . NVD_API_KEY }

- name: Build Docker picture

run: docker build -t digibank:ci .

- name: Run Trivy on Docker picture

uses: aquasecurity/trivy-action@0.24.0

with:
```

```
image-ref: digibank:ci

format: table

severity: HIGH, CRITICAL

exit-code: '1'

ignore-unfixed: true

- name: Upload Dependency-Check report

  uses: actions/upload-artifact@v4

  with:

    name: dependency-check-report

    path: target/dependency-check-report.html
```

This workflow directly extends the practices already introduced in the workshop. It rebuilds DigiBank, performs the Maven dependency scan, produces the Docker image, and then submits this image to Trivy before validating the rest of the chain. Thus, security is no longer limited to the internal Java project, but extends to the containerized artifact actually intended for execution or deployment. [Attachment +1]

It's also important to remember that secrets needed for the pipeline, such as the NVD API key, should never be hard-coded in the repository. The course emphasizes the importance of handling sensitive information and separating application artifacts from confidential data. Therefore, in GitHub Actions, these elements should be stored in the repository's Secrets and then injected at runtime. [Attachment +1]

5.6 Maven adjustments and recommended configuration

Lab 2 already demonstrates that Maven is the natural entry point for orchestrating controls in DigiBank. Therefore, it is recommended to keep security plugins at the parent pom.xml level to maintain consistent execution locally, within the IDE, and in GitHub Actions.

Following on from previous workshops, the important properties of the Maven parent must remain centralized:

```
< properties >

  < java.version > 17 </java.version>

      < maven.compiler.plugin.version > 3.13.0
</maven.compiler.plugin.version>

      < dependency-check.version > 10.0.4 </
dependency-check.version>

  < surefire.version > 3.2.5 </surefire.version>

< /properties >
```

This centralization is consistent with the spirit of TP2, which emphasizes the readability and maintainability of the multi-module configuration. It avoids scattering versions across multiple modules and facilitates future updates to the security pipeline. [Attachment]

The OWASP Dependency-Check plugin can be kept in the parent pom.xml file with a configuration like this:

```
<plugin>

  <groupId> org.owasp </groupId>

  < artifactId > dependency-check-maven </artifactId>

  < version >${ dependency-check . version }</version>

  < configuration >
```

```

<format> HTML </format>

<failBuildOnCVSS> 7 </failBuildOnCVSS>

< skipProvidedScope > true </skipProvidedScope>

< skipTestScope > true </skipTestScope>

< nvdApiKey >${ approx . NVD_API_KEY }</nvdApiKey>

</configuration>

< executions >

  < execution >

    <phase> verify </phase>

    <goals>

      <goal> check </goal>

    < /goals >

  </execution>

< /executions >

</plugin>

```

This configuration follows the logic already presented in Lab 2: generate an HTML report, fail the build beyond a certain criticality threshold, and use an NVD key injected via an environment variable rather than hard-coded. This last requirement is particularly important, as the course emphasizes that secrets should not be left in versioned files.

It is also advisable to maintain an explicit Surefire configuration to stabilize test execution:

```
<plugin>

  <groupId> org.apache.maven.plugins </groupId>

  < artifactId > maven-surefire-plugin </artifactId>

  < version >${ surefire . version }</version>

  < configuration >

    <useModulePath> false </useModulePath>

  </configuration>

</plugin>
```

This precaution improves the reproducibility of tests between the local environment and GitHub Actions. Following the logic of Lab 2, this type of setting prevents Maven execution problems from being mistaken for genuine security or quality signals.

5.7 Pipeline Secrets and Variables Management

A secure CI/CD pipeline should never introduce new weaknesses by exposing secrets directly in the repository. Both the lecture and lab 2 emphasize this point: passwords, access tokens, or API keys visible in configuration files are themselves potential vulnerabilities. [Attachment +1]

In GitHub Actions, the key required for OWASP Dependency-Check must therefore be created in the repository secrets, for example under the name:

```
NVD_API_KEY
```

Once this secret is recorded in GitHub, it can be injected into the workflow using the env block. This method respects the separation between pipeline configuration and sensitive data, while remaining compliant with the DevSecOps logic presented in the course.

The same principle applies to other future integrations, such as a SonarQube token, a private container registry, or deployment credentials. A good pedagogical practice is to explicitly state in the workshop document that a GitHub repository should never contain values like `NVD_API_KEY=...`, `SONAR_TOKEN=...`, or Docker registry credentials in plain text.

5.8 Local verification before pushing to GitHub

Before pushing the workflow to GitHub, it's recommended to replay the main pipeline commands locally. This step remains true to the philosophy of previous workshops: a reliable pipeline relies first and foremost on a locally understandable and executable project.

A typical sequence can be presented as follows:

```
CD digibank-parent

mvn clean

mvn test

mvn clean verify

mvn dependency:tree > dependency-tree.txt

mvn                    org.owasp:dependency-check-maven:check
-DnvdApiKey=YOUR_NVD_API_KEY

docker build -t digibank:local .

docker                picture                inspect                digibank:local                >
docker-image-inspect.json

docker                history                digibank:local                --no-trunc                >
docker-image-history.txt
```

This sequence allows us to verify several aspects before any remote execution: compilation, test stability, proper generation of the dependency tree, functionality of the OWASP scan, and the project's actual ability to produce its Docker image. It also embodies the repeatability principle central to Lab 2, where the same checks must be understandable and replayable locally, then transposed almost identically to GitHub Actions.

The student must then examine several elements. First, the Maven build must not fail without an identified cause; second, the Dependency-Check report must be generated correctly; and finally, the Docker image must be built without Dockerfile errors or missing dependencies. If any of these points fail locally, they must be corrected before the CI/CD pipeline can be considered ready.

5.9 Push, GitHub Actions Execution and Artifact Control

Once the local checks are validated, the workflow can be added to the repository and then pushed to GitHub:

```
git add .github/workflows/digibank-security-pipeline.yml
git commit -m "Add CI/CD security pipeline for DigiBank"
git push origin develop
```

From this point on, GitHub Actions should automatically trigger the workflow on the relevant branch, according to the rules defined in the YAML file. This automation directly extends the logic of Lab 2, where GitHub Actions is presented as the natural support for industrializing builds, tests, and security checks.

After execution, several points must be checked in the GitHub interface. The job must appear as successfully completed, or failing that, fail in an interpretable way; the logs must show the correct execution of Maven, OWASP Dependency-Check and Docker; finally, the artifacts must be downloadable.

The main artifacts expected in this version of the pipeline are as follows:

- `dependency-tree.txt`, to observe the actual dependency structure.
- `dependency-check-report.html`, to examine the known vulnerabilities detected.
- `docker-image-inspect.json`, to store metadata about the constructed image.
- `docker-image-history.txt`, to trace the image construction layers.

These artifacts play an important role in knowledge capitalization. They not only allow us to verify that a control has been carried out, but also to compare the results over time, which aligns with the logic of traceability and technical proof already valued in the final validation of TP2.

5.10 Security CI/CD Chain Acceptance Criteria

A corrected version of DigiBank can be considered acceptable at the end of the workshop if several criteria are met simultaneously. The first criterion is the absence of major functional regressions. A more secure application that has become unstable or incorrect from a business perspective would not meet the learning objectives of the exercise. Security should enhance the solution, not invalidate its expected behavior.

The second criterion is the effective reduction of prioritized vulnerabilities. If critical alerts identified in previous analyses are still present without a substantiated explanation, remediation cannot be considered complete. This applies both to dependencies detected by OWASP Dependency-Check and to vulnerabilities reported by Trivy on DigiBank's Docker image.

The third criterion is the architectural consistency of the fixes. Remediation must respect DigiBank's modular organization, integrate cleanly into DTOs, services, controllers, configurations, and containerization artifacts, and not create new security debt under the guise of addressing an existing vulnerability. A high-quality fix is therefore effective, clean, maintainable, and compatible with pipeline industrialization.

The fourth criterion is the reproducibility of checks. A truly acceptable version is not just one that passes a one-time manual check, but one whose state can be reconfirmed by clear commands, by Maven, by Docker, by Dependency-Check, and by Trivy, both locally and in GitHub Actions. This requirement aligns precisely with the DevSecOps logic of traceability, automation, and continuous validation.

5.11 Expected outcome of Part 5

The workshop's conclusion must also include a knowledge-sharing component. In a DevSecOps approach, it's not enough to simply fix issues; a clear record of what has been corrected, what remains under observation, and the reasons behind the decisions made is essential. The practical exercise already highlights Git, Maven, and GitHub Actions as tools for reproducibility and traceability, and the addition of Trivy naturally follows this same documentation logic.

This capitalization can take several complementary forms:

- Explicit Git commits, reflecting remediation intentions.
- A technical note or workshop report describing the vulnerabilities observed, the actions taken, and the results obtained.
- Retention of Dependency-Check reports and, where useful, Trivy reports.
- An update to the GitHub Actions pipeline so that future changes continue to be evaluated automatically.

At the end of this section, the student should not only have a locally remedied DigiBank project, but should have transformed it into a system capable of automatically replaying its main security checks with each significant change to the code, dependencies, or Dockerfile.

In concrete terms, DigiBank must now be able to rebuild itself in GitHub Actions, run its tests, analyze its dependencies, produce its container image, and publish searchable artifacts related to these operations. This progression corresponds exactly to the overall logic of the course: moving security upstream, integrating it into the CI/CD pipeline, and making it continuous, traceable, and actionable. [Attachment]

From a pedagogical standpoint, Part 5 marks a significant shift in approach. Security is no longer simply an object of local analysis or correction; it becomes a progressively industrialized property of the DigiBank project, integrated into its software development pipeline and ready to support the subsequent stages of the DevSecOps journey.



Part 6: Final validation, capitalization, and preparation for the next step

After identifying vulnerabilities, remediating containers, updating dependencies, and integrating initial controls into the CI/CD pipeline, the workshop must conclude with a final validation and consolidation phase. This step is essential because, in a DevSecOps approach, a fix is only truly useful if it is verified, documented, and integrated into a continuous security process rather than considered an isolated action.

In the case of DigiBank, this final section therefore serves a threefold purpose. First, it must confirm that the measures taken have indeed improved the project's security posture without disrupting its overall operation. Second, it must document the results obtained to maintain a usable record of the decisions, remediation efforts, and technical trade-offs. Finally, it must prepare for the next steps by demonstrating that the resulting version now provides a more credible foundation for the subsequent stages of security, automation, and continuous monitoring.

From a pedagogical standpoint, this section aligns with the overall approach of the course. The course emphasizes that security should not be viewed as a single, end-user control, but rather as a continuous requirement integrated into the pipeline, dependencies, delivery artifacts, and execution environments. The final validation, therefore, does not simply "end" the workshop; it verifies that DigiBank has genuinely progressed from a more exposed initial state to one that is better controlled, more traceable, and more easily scalable.

6.1 Role of final validation

The main purpose of final validation is to demonstrate that the corrections made in the workshop are not simply local or opportunistic adjustments. It must show that the changes made to dependencies, containers, and the pipeline have been properly integrated,



that they effectively mitigate certain risks, and that they leave behind a technically sound project. This logic directly aligns with that already established in Workshop 2, where security is considered complete only after a re-verification of the compilation, tests, analysis reports, and overall project consistency.

From a DevSecOps perspective, this step also allows us to move from a logic of one-off fixes to a logic of control. Fixing a Dockerfile, updating a dependency, or adding a control in GitHub Actions only has lasting value if we can then prove that the system rebuilds correctly, that the resulting artifacts are cleaner, and that the results obtained can be replayed or observed again. The final validation therefore functions as a confirmation mechanism, but also as a first level of technical security governance.

Finally, it is important because remediation can introduce side effects. A dependency update can break compatibility, image hardening can remove a useful component, and a stricter pipeline can cause integration to fail for reasons that must be interpreted. Final validation, therefore, serves not only to confirm an improvement; it also ensures that this improvement remains compatible with the stability, maintainability, and future evolution of DigiBank.

6.2 What needs to be checked

Following the remediation work carried out in the workshop, several aspects must be explicitly rechecked. The first concerns the overall technical stability of the project. DigiBank must still be able to be correctly rebuilt from its root, the Maven modules must remain consistent, and the normal build cycle must continue to function despite the updates and security adjustments introduced in previous sessions. This requirement is consistent with previous workshops, which emphasize that a secure project must remain functional, testable, and ready for deployment.

The second dimension concerns the security artifacts themselves. Dependencies previously identified as vulnerable must have been updated, replaced, or justified; the Docker image must reflect a reduced attack surface or a better level of control; and reports from the tools integrated into the pipeline must indicate a cleaner or at least better-documented situation. This is not about requiring a state completely free of all alerts, but about verifying that the risks deemed priorities have been addressed and no longer exist in their original form.



The third dimension concerns the consistency of the CI/CD pipeline. The workflow added in the previous section must trigger correctly, rebuild the project, execute the expected checks, produce the reporting artifacts, and make the results visible in a usable way. The course emphasizes that modern security must be integrated into the software manufacturing pipeline, and not left to late manual checks. Revalidating the pipeline therefore means verifying that security has been properly integrated into the normal operation of the project.

Finally, a fourth dimension must be considered: the quality of interpretation. In a DevSecOps framework, a report or scan is worthless if it is not understood. Students must therefore be able to explain what has been corrected, what has been improved, what may still be under monitoring, and why certain technical decisions were made. This ability to justify is directly consistent with the course's approach, which repeatedly emphasizes that security is never simply about the automated generation of alerts.

6.3 Expected technical checks

Final validation must rely on a clear sequence of technical checks. The project must first be rebuilt cleanly from its root to verify that changes made to dependencies, the Dockerfile, and the pipeline have not broken the overall build consistency. This verification follows the Maven logic already highlighted in previous workshops as the primary entry point for technical reproducibility.

Existing tests must then be rerun to ensure that no major functional regressions have been introduced. While Workshop 4 focuses more on dependencies, containers, and automated checks, it remains essential to remember that useful security measures should not invalidate the application's expected behavior. A "safer" version that has become unstable or inconsistent from a business perspective would not satisfy the learning objectives of the course.

The security tools already in use must also be restarted after the fix. OWASP Dependency-Check should be used to verify whether the problematic dependencies have indeed been removed from the report, given their reduced criticality, or were reasonably justified if an immediate update was not possible. Meanwhile, the GitHub Actions pipeline should be rerun to confirm that the build, dependency scan, image build, and artifact publishing are all executing correctly in an automated environment.

A typical final validation sequence can thus be presented in a simple way:

```
mvn clean

mvn test

mvn clean verify

mvn org.owasp:dependency-check-maven:check

docker build -t digibank:final .
```

If the GitHub Actions workflow has been configured according to the previous section, the verification can be extended by a push or pull request that automatically triggers the pipeline. In this case, the validation is not limited to the student's local environment: it is part of a logic of reproducibility and continuous monitoring, perfectly in line with the spirit of DevSecOps presented in the course.

6.4 Criteria for accepting a corrected version

A corrected version of DigiBank can be considered acceptable at the end of the workshop if several criteria are met simultaneously. The first criterion is the absence of major technical regressions. The application must still build correctly, its components must remain consistent, and any modifications introduced for security reasons must not have degraded the overall functionality of the project.

The second criterion is the effective reduction of priority risks. Dependencies deemed problematic must have been addressed, major containerization flaws must have been corrected or mitigated, and the pipeline must now incorporate controls that make these elements visible with each significant change. The course emphasizes that the security of containers and dependencies must be monitored over time, as these artifacts evolve and can become risky again if no continuous verification mechanism is in place.

The third criterion is the architectural and operational consistency of the remediation measures. Corrections must not create new debt under the pretext of reducing existing debt. A good remediation is therefore not only effective; it is also readable, maintainable, compatible with the project, and integrable into its workflow.



The fourth criterion is traceability. Changes made must be traceable in the Git history, useful reports must remain accessible, and technical choices must be explainable. This traceability requirement is consistent with the overall DevSecOps logic of the course, which closely links security, reproducibility, automation, and software lifecycle governance.

6.5 Capitalization of Results

The workshop's conclusion must also explicitly include a knowledge-sharing component. In a DevSecOps approach, it's not enough to simply fix things; a clear record of what was observed, what was decided, what was modified, and what might require further monitoring is essential. Workshop 2 already emphasized the importance of Git, Maven, and GitHub Actions as tools for traceability and reproducibility; this logic remains fully valid here.

This knowledge capitalization can take several complementary forms. First, commits should reflect understandable intentions, such as a dependency update, a Dockerfile hardening, or the addition of a CI/CD check. Second, reports generated by Dependency-Check and pipeline-related artifacts should be retained as evidence of the achieved state. Finally, the workshop report itself should explain the initial problems, the chosen remediations, the results after correction, and any remaining limitations.

From a pedagogical standpoint, this capitalization has significant value. It helps students shift from a tool-execution mindset to a structured analytical approach. It also demonstrates that safety is not just about instant diagnosis, but also about technical documentation, justification, and continuous improvement. A project that properly capitalizes on its remediation becomes easier to revisit, audit, and adapt in subsequent workshops.

6.6 Preparing for the next part of the journey

The final function of this section is to explicitly prepare for what comes next. The course presents DevSecOps as a continuous process, in which controls are sequential and complementary rather than canceling each other out or replacing one another. Securing



dependencies, hardening containers, and integrating an initial security CI/CD pipeline are therefore not ends in themselves; they form a cleaner foundation for the next stages of project development.

In the case of DigiBank, this progression means that the version produced at the end of the workshop is no longer just a functional application enhanced with a few local fixes. It becomes a more credible system for future analysis because it relies on better-managed dependencies, a more consistent container image, and a pipeline capable of replicating some of the essential controls. This aligns perfectly with the course's logic, which demonstrates that security must be considered in the code itself, in the accompanying artifacts, and in the entire chain that generates them.

Thus, preparing for the next phase is not simply about “moving on to the next workshop,” but about demonstrating that DigiBank now has a more robust foundation to support more advanced controls, safer upgrades, and the gradual industrialization of security. The final validation, knowledge sharing, and preparation for the next phase together form a logical conclusion to the workshop: they transform a series of ad-hoc fixes into a more sustainable, observable, and DevSecOps-compliant working basis.